Fase 5 — Pesaje final y conciliación (Punto 5)

Rama: `fase-5-conciliacion` · Implementación y pruebas: 02/11/2026 · Parte 1: **Tomás Calvo** · Parte 2: **Benjamín Alasino** · Estado: **completa y verificada en la rama** (observaciones y pendientes en las secciones 10 y 11).

Este archivo tiene dos partes:

- **Parte 1:** informe técnico de lo implementado, lo verificado y la auditoría final del segundo parcial.
- **Parte 2:** guía de estudio para la defensa oral (secciones A–J, más la K: qué debe poder explicar cada integrante).

Convención de rutas: `SRC` = `src/main/java/ar/edu/iua/iw3`. La división del trabajo, el contrato (decisiones D1 a D14) y los registros de cada parte están en `docs/fase-5-division-de-trabajo.md`. La fuente de los requisitos es `docs/pdf/consigna-trabajo-integrador-2026.pdf`.

Parte 1 — Informe técnico

1. Alcance y resultado

La fase implementa el **Punto 5 de la consigna**: "*el sistema de balanza (TMS) envía el pesaje final del camión. El backend debe contar con un servicio que reciba este pesaje final, cambie la orden a estado 4: Finalizada, y devuelva una conciliación con los valores calculados [...] La conciliación podrá solicitarse en cualquier momento, pero sólo para órdenes en estado 4, a través de un endpoint específico.*"

- **Endpoint nuevo de la balanza:** `POST /api/v1/integration/tms/orders/{orderNumber}/final-weight` con `{ "weight": 16490.0 }`. Pasa la orden de 3 a 4, guarda el peso y la hora de recepción, registra el cambio en el historial y responde **200 con la conciliación**.
- **Endpoint nuevo de consulta:** `GET /api/v1/orders/{orderNumber}/reconciliation`. Devuelve la misma conciliación, **solo en estado 4** (otro estado: 409).
- **Dos arreglos de fases anteriores** autorizados (D13 y D14): la tara rechaza un peso infinito con 400 (H-26) y la activación busca la orden con bloqueo de fila (H-25; resuelve también H-16).
- **Cierre del segundo parcial:** carpeta Postman 05 (Punto 5), carpeta 06 (circuito completo 1 → 4 con *Tests* encadenados), pruebas de concurrencia y **auditoría final** contra el PDF oficial (sección 9).

Resultado: los **21 criterios** de aceptación de la sección 6 de la división están cumplidos y verificados con pruebas ejecutadas (tabla abajo). Los requisitos obligatorios del segundo parcial para los Puntos 1 a 5 quedan **implementados y verificados**. La auditoría registra tres observaciones nuevas (H-27 a H-29); ninguna afecta a los Puntos 1 a 5 ni bloquea la fase.

#	Criterio de aceptación (división, sección 6)	Evidencia	Resultado
1	Pesaje final válido: 200 con la conciliación y estado 4 (también en `GET /orders/{n}`)	Postman 05 «Pesaje final (estado 3 → 4)» y «Consultar la orden finalizada»; 06 «Punto 5»	Cumplido
2	`finalWeight` y `finalWeighingDateTime` guardados y visibles (hora del servidor)	Postman 05 «Consultar la orden finalizada»; 06 «Punto 5: la orden queda en estado 4»	Cumplido
3	Una fila 3 → 4 con usuario y observación; el circuito deja 4 filas	Postman 05 «Historial de la orden finalizada»; 06 «Historial completo» (`null→1`, `1→2`, `2→3`, `3→4`)	Cumplido
4	Fórmulas: `loadedProduct` 2000.0, `scaleNet` 1990.0, `difference` −10.0	Postman 05 (`closeTo`); `OrderBusinessFinalWeightTest`	Cumplido
5	Promedios = `AVG` de las lecturas (21.0, 0.5, 3300.0)	Postman 05; 06 los recalcula en el test con otros valores	Cumplido

#	Criterio de aceptación (división, sección 6)	Evidencia	Resultado
6	Sin lecturas: 200, `loadedProduct` 0.0 y promedios `null`	Postman 05 «Pesaje final igual a la tara (sin lecturas)»	Cumplido
7	Peso ausente, 0, negativo o infinito: 400; menor que la tara: 400; igual a la tara: 200	Postman 05 (5 pedidos 400 y el caso límite 200)	Cumplido
8	Un 400 no cambia nada	Postman 05: estado 3, `finalWeight` y fecha nulos, **3** filas de historial	Cumplido
9	Estados 1, 2 y 4: 409 · segundo pesaje: 409 · inexistente: 404 · sin token: 401	Postman 05	Cumplido
10	`GET …/reconciliation` : estado 4 = POST; estados 1, 2 y 3: 409; inexistente 404; sin token 401	Postman 05 y 06 (comparación campo a campo con el POST)	Cumplido
11	En estado 4: lecturas, activación, cierre, tara y pesaje final dan 409	Postman 05 «Estado 4 es final» (5 pedidos)	Cumplido
12	Dos pesajes finales simultáneos: un 200 y un 409, una sola fila 3 → 4	Concurrencia (a), 5 repeticiones	Cumplido
13	H-26: tara `1e999` → 400, la orden sigue en estado 1 con tara nula	Postman 05 «Regresión H-26»; `OrderBusinessTareTest`	Cumplido
14	H-25: activación y cierre simultáneos nunca dejan el estado 2; dos activaciones guardan la fecha una vez	Concurrencia (b) 20 repeticiones y (c) 5 repeticiones	Cumplido
15	Las columnas nuevas se agregan sobre una base con órdenes	Parte 1 (clon de `iw3_db`); `iw3_db` hoy: `final_weight` y `final_weighing_date_time` nulables, orden previa intacta	Cumplido
16	Swagger documenta los dos endpoints con sus respuestas	`/v3/api-docs` : `final-weight` 200/400/404/409, `reconciliation` 200/404/409, esquema `Reconciliation`	Cumplido (ver H-27: no declaran 401 ni 500)
17	Carpetas 05 y 06 pasan y las 00 a 04 siguen (86 pedidos, 204 aserciones)	newman: 151 pedidos, 358 aserciones, 0 fallas	Cumplido
18	Tests nuevos y `./mvnw -B test` en verde	40 tests, 0 fallas	Cumplido
19	La carpeta 06 simula el circuito 1 → 4 encadenado	21 pedidos, 56 aserciones; corre sola (con su login)	Cumplido
20	Auditoría final contra el PDF	Sección 9	Cumplido
21	Informe, guía (≥ 10 preguntas), `CLAUDE.md` , plan, guía de `src` y PDF	Este documento (16 preguntas) y los commits `docs` de la Parte 2	Cumplido

2. Decisiones de diseño (D1 a D14 = decisiones 33 a 46)

Confirmadas por Martín González el 02/11/2026. La numeración global (33 a 46) es la de `CLAUDE.md` y del plan.

D	#	Decisión	Por qué	Alternativa descartada
D1	33	`final-weight` solo desde el **estado 3**; 1, 2 y 4 dan **409**; no es idempotente	Estados secuenciales y unidireccionales (consigna, notas). Un segundo pesaje no es un reintento inocuo: cambiaría el peso y sumaría otra fila	Idempotente (200 la segunda vez): ocultaría un error de la balanza y el segundo peso podría ser distinto
D2	34	Orden **404** → **400** → **409** → **400**: orden; peso ausente, no finito o ≤ 0; estado; peso **menor** que la tara. Igual a la tara es válido	La comparación con la tara necesita la tara, que es nula en el estado 1 (sería un `NullPointerException` → 500). Peso igual: una orden cerrada sin cargar pesa lo mismo que vacía	Comparar con la tara antes del estado (500 en el estado 1); rechazar `weight <= tare` (la orden sin lecturas quedaría trabada en el estado 3)

D	#	Decisión	Por qué	Alternativa descartada
D3	35	`findByOrderNumberForUpdate`, comprobaciones fuera del `try`, `@Transactional(rollbackFor = Exception.class)`, `saveAndFlush`. El GET no bloquea	Mismo molde que el cierre (decisión 31): dos pesajes simultáneos se serializan y el segundo ve el estado 4	Bloqueo optimista con `@Version`: obliga a reintentar y es un cambio de modelo de la Fase 1
D4	36	Se guardan solo `finalWeight` y `finalWeighingDateTime` (hora del servidor). La conciliación **no se guarda**	Se deriva de datos ya guardados; guardarla duplicaría información que podría quedar desalineada. La consigna define la fecha como "momento de recepción"	Columnas para los valores calculados, o fecha enviada por el cliente
D5	37	Producto cargado = `lastAccumulatedMass`; neto = `finalWeight − tare`; diferencia = neto − producto; promedios = `AVG` simple. **Sin redondeo**	Son las fórmulas literales del PDF; "según datos almacenados" = las filas de `order_details`	Promedio ponderado por tiempo (la consigna no lo pide); redondear en el backend (el cliente formatea)
D6	38	Orden sin lecturas: producto cargado **0.0**, promedios `null`; no bloquea el pesaje final	Posible por la decisión 30 (cierre sin lecturas). Rechazarla dejaría la orden en el estado 3 sin salida	Exigir lecturas (409) o promedios 0.0 (un 0 de temperatura sería un dato falso)
D7	39	El POST responde 200 con la conciliación (el mismo objeto que el GET); GET solo en estado 4	"Devuelva una conciliación" (Punto 5)	Responder la orden y obligar a otro GET
D8	40	Una fila 3 → 4, usuario del token, `Pesaje final registrado`. El estado 4 es final	Registro de cada cambio de estado (consigna, notas)	—
D9	41	Lógica en `IOrderBusiness` / `OrderBusiness` con `OrderDetailRepository` inyectado directo; POST en `OrderTmsRestController`, GET en `OrderRestController`; `Constants` sin cambios	`OrderDetailBusiness` ya depende de `IOrderBusiness`: inyectarlo sería circular y el contexto no arrancaría	`IOrderDetailBusiness` en `OrderBusiness`; un controller nuevo
D10	42	Mensajes con el estilo de las fases anteriores; Postman compara fragmentos	Un cambio de redacción no rompe las pruebas	Comparar el texto completo
D11	43	Carpetas 05 y 06 con variables propias; las 00 a 04 no se tocan	Carpetas independientes; línea base 86/204/19	Reutilizar `orderNumber`, `orderNumberClose`, etc.
D12	44	Fuera de alcance: frecuencia de almacenamiento, roles, alarmas, frontend, seguridad	Opcional del parcial o examen final	—
D13	45	Tara con peso infinito → **400** con `requireValidWeight` (compartido con el pesaje final); el mensaje de ≤ 0 no cambia	`1e999` pasaba `!(w > 0)` y MySQL lo rechazaba al guardar: 500 con el formato de Spring (H-26)	Un manejador global de errores (cambio transversal)
D14	46	`activate` busca con `loadForUpdate` después de comprobar el cuerpo	Una primera activación simultánea con el cierre podía reescribir el estado 2 (H-25); también serializa dos activaciones (H-16)	Dejarlo aceptado como en la Fase 4

3. Archivos de la fase

Atribución. La columna «Responsable asignado» sale de la división del trabajo; la columna «Commits» es lo observado en `git log` (autor `tomas-calvo01` para la Parte 1 y `Benjamin Alasino` para la Parte 2). Coinciden en todos los casos.

3.1 Parte 1 — Tomás Calvo (negocio y endpoints)

Archivo	Acción	Motivo	Commits
`SRC/model/Order.java`	Modificado	`finalWeight` (`Double`) y `finalWeighingDateTime` (`LocalDateTime`), opcionales y `READ_ONLY`	`4131c00`
`SRC/model/persistence/ReadingAverages.java`	Creado	Proyección con los tres promedios	`ff05d96`
`SRC/model/persistence/OrderDetailRepository.java`	Modificado	`averagesByOrder(Order)` con `AVG`	`ff05d96`
`SRC/business/Reconciliation.java`	Creado	Respuesta de la conciliación (9 campos, `@Schema`)	`6742511`
`SRC/business/OrderBusiness.java`	Modificado	`requireValidWeight` y su uso en `registerTare` (D13); `loadForUpdate` en `activate` (D14); `registerFinalWeight` , `reconciliation` , `buildReconciliation` ; inyección de `OrderDetailRepository`	`8ea3244` , `59a79b9` , `eb5a203` , `6a876a8`
`SRC/business/IOrderBusiness.java`	Modificado	Firmas de `registerFinalWeight` y `reconciliation` con Javadoc	`eb5a203` , `6a876a8`
`SRC/integration/tms/model/FinalWeightRequest.java`	Creado	DTO con `double weight`	`867f4a9`
`SRC/integration/tms/controllers/OrderTmsRestController.java`	Modificado	`POST /{orderNumber}/final-weight`	`6946d85`
`SRC/controllers/OrderRestController.java`	Modificado	`GET /{orderNumber}/reconciliation`	`7408b07`
`src/test/java/.../OrderBusinessTareTest.java`	Creado	4 tests (H-26)	`6a1a558`
`src/test/java/.../OrderBusinessActivateTest.java`	Creado	6 tests (H-25)	`d4274f7` , `a5e76aa`
`src/test/java/.../OrderBusinessFinalWeightTest.java`	Creado	11 tests del pesaje final y la conciliación	`f57dc6c` , `7e1fab3` , `1bd80d0` , `736f027`
`docs/guia-estudio-src.md` , `docs/fase-5-division-de-trabajo.md`	Modificados	Filas F5 y bloque "Estado de la Parte 1"	`f16930d` , `9347a95`

3.2 Parte 2 — Benjamín Alasino (pruebas, auditoría y documentación)

Archivo	Acción	Motivo
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Modificado (solo agrega)	Carpetas 05 (44 pedidos, 98 aserciones) y 06 (21 pedidos, 56 aserciones) y 16 variables propias. `git diff --numstat` : 3142 líneas agregadas, 0 borradas (`0622a20`)
`docs/fase-5-conciliacion.md` y `docs/pdf/fase-5-conciliacion.pdf`	Creados	Este informe y su guía
`CLAUDE.md` , `docs/plan-de-trabajo.md` , `docs/guia-estudio-src.md` , `docs/pdf/README.md`	Modificados	Estado del proyecto, decisiones 33 a 46, hallazgos y mapa de código (apartado F de la división)
`docs/fase-5-division-de-trabajo.md` y su PDF	Modificados	Bloque "Estado de la Parte 2"

La Parte 2 **no modificó** código de `src/main` ni de `src/test`. Los scripts de concurrencia y el generador de las carpetas Postman fueron descartables y quedaron fuera del repo.

No se tocaron: `Constants`, `OrderDetailBusiness`, `OrderDetailValidator`, `OrderRepository`, `OrderStatus`, `SecurityConfiguration`, los `.properties` ni el `pom.xml` (sin dependencias nuevas). `git diff origin/main..HEAD` muestra solo los archivos de las tablas anteriores.

4. Recorrido de `POST /api/v1/integration/tms/orders/{orderNumber}/final-weight`

1. **Filtro JWT** (`JWTAuthorizationFilter`). Sin token o con uno inválido: **401**; no llega al controller.
2. **Controller** — `OrderTmsRestController.registerFinalWeight(@PathVariable Integer orderNumber, @RequestBody FinalWeightRequest body)`. `consumes` y `produces` JSON: otro `Content-Type` da 415; sin cuerpo, 400 de Spring; `{orderNumber}` no numérico o `weight` no numérico, 400 de Spring (H-21, H-23). Jackson convierte `{}` en `weight = 0.0` (primitivo) y `1e999` en `Infinity`. Toma el usuario con `getUserLogged()` y llama a `orderBusiness.registerFinalWeight(orderNumber, body.getWeight(), username)`.
3. **Negocio** — `OrderBusiness.registerFinalWeight`, en una transacción (`@Transactional(rollbackFor = Exception.class)`). Todas las comprobaciones **fuera del** `try`:
   1. `loadForUpdate(orderNumber)` → `SELECT … FOR UPDATE`: **bloquea la fila**. No existe → `NotFoundException` (**404**).
   2. `requireValidWeight(weight)`: `!(weight > 0)` → **400** `weight debe ser mayor que 0 (kg)` (incluye `NaN`); infinito → **400** `weight debe ser un número finito`.
   3. Estado distinto de `CLOSED_FOR_LOADING` → **409** `La orden N no admite pesaje final: debe estar en estado 3 (estado E)`.
   4. `tare` nula → **409** `La orden N no tiene tara registrada` (no debería pasar en el estado 3).
   5. `weight < tare` → **400** `weight (pesaje final) no puede ser menor que la tara (T kg)`.
   6. Dentro del `try`: `from = 3`; `finalWeight = weight`; `finalWeighingDateTime = now()`; estado `FINISHED`; `orderDAO.saveAndFlush(order)` (`UPDATE orders`); `logStatus(order, 3, 4, actor, "Pesaje final registrado")` (`INSERT order_status_logs`); `buildReconciliation(order)` (`SELECT avg(…) FROM order_details …`).
   7. Una falla técnica en el paso 6 → `BusinessException` (500) y *rollback* de las dos escrituras.
4. **`COMMIT`:** se libera la fila. Un segundo pesaje que esperaba el bloqueo lee ahora el estado 4 y recibe 409.
5. **Respuesta:** 200 con la `Reconciliation`. Errores con `response.build(status, e, e.getMessage())` (`StandartResponse`): 400, 404, 409, 500.

5. Recorrido de `GET /api/v1/orders/{orderNumber}/reconciliation`

1. Filtro JWT (401 sin token).
2. `OrderRestController.reconciliation(@PathVariable Integer orderNumber)` → `orderBusiness.reconciliation(orderNumber)`.
3. `OrderBusiness.reconciliation` (sin `@Transactional` ni bloqueo: solo lee): `load(orderNumber)` → 404; estado distinto de `FINISHED` → **409** `La conciliación solo está disponible para órdenes en estado 4 (orden N en estado E)`; en el `try`, `buildReconciliation(order)` (un error técnico → `BusinessException`, 500).
4. 200 con la misma estructura que devolvió el POST. Como ambos usan **la misma función** sobre los mismos datos guardados, los valores coinciden (verificado campo a campo en Postman 05 y 06).

6. Funciones y métodos

`OrderBusiness.registerFinalWeight` — `SRC/business/OrderBusiness.java`

Firma	`Reconciliation registerFinalWeight(Integer orderNumber, double weight, String actor) throws NotFoundException, BadRequestException, ConflictException, BusinessException`
Parámetros	`orderNumber` (path), `weight` (kg, del cuerpo), `actor` (usuario del token; puede ser `null`)
Retorno	La conciliación calculada
Validaciones	404 → 400 (peso) → 409 (estado 3) → 409 (tara nula) → 400 (menor que la tara), todas fuera del `try`
Escribe	`orders.final_weight`, `orders.final_weighing_date_time`, `orders.status = 4` y una fila en `order_status_logs`
Dependencias	`orderDAO` (con bloqueo), `statusLogDAO` (vía `logStatus`), `orderDetailDAO` (vía `buildReconciliation`)

`OrderBusiness.reconciliation`

`Reconciliation reconciliation(Integer orderNumber) throws NotFoundException, ConflictException, BusinessException`. 404 → 409 (estado ≠ 4) → `buildReconciliation`. No escribe ni bloquea.

`OrderBusiness.buildReconciliation` (privado)

La **única** función que arma la conciliación (D4). Sin redondeo:

Campo	Cálculo
`orderNumber`, `tare`, `finalWeight`	De la orden
`loadedProduct`	`lastAccumulatedMass`, o `0.0` si es nulo (D6)
`scaleNet`	`finalWeight − tare`
`difference`	`scaleNet − loadedProduct`
`averageTemperature`, `averageDensity`, `averageFlowRate`	`orderDetailDAO.averagesByOrder(order)`; `null` si la proyección es `null` o si sus valores lo son (cero lecturas)

`OrderBusiness.requireValidWeight` (privado, D13)

`!(weight > 0)` → 400 `weight debe ser mayor que 0 (kg)`; `Double.isInfinite(weight)` → 400 `weight debe ser un número finito`. La usan `registerTare` y `registerFinalWeight`.

`OrderBusiness.activate` (cambio D14)

Igual que en la Fase 3 salvo una línea: después de comprobar `orderNumber` / `password` nulos (400), busca con `loadForUpdate` en lugar de `load`. El orden 400 → 404 → 409 → 403, el mensaje del 403 (sin contraseñas) y la idempotencia no cambiaron.

`OrderDetailRepository.averagesByOrder` y `ReadingAverages`

`@Query("select avg(d.temperature) as temperature, avg(d.density) as density, avg(d.flowRate) as flowRate from OrderDetail d where d.order = ?1")`. Los alias coinciden con los getters de la interfaz de proyección `ReadingAverages` (`getTemperature`, `getDensity`, `getFlowRate`). Con cero lecturas el `AVG` de SQL devuelve una fila con `NULL`: la API responde los tres promedios `null` (verificado con pedidos reales).

`Reconciliation`, `FinalWeightRequest` y los controllers

- `Reconciliation` (`business`): `@Getter`, `@AllArgsConstructor`, 9 campos con `@Schema` y los ejemplos de la sección 3.5 de la división.
- `FinalWeightRequest` (`integration/tms/model`): `double weight` primitivo, sin fecha.

- `OrderTmsRestController.registerFinalWeight` y `OrderRestController.reconciliation`: `try/catch` por excepción, `@Operation` y `@ApiResponse` (200 con esquema `Reconciliation`, 400/404/409 y 404/409).

7. Pruebas ejecutadas (02/11/2026)

Entorno: Windows 11, JDK 21, MySQL 8.0 en Docker (`iw3-gas-mysql`), Node.js 26.13 (fuera del `PATH`; se agregó `C:\Program Files\nodejs` en cada comando). Las pruebas de API de la Parte 2 se hicieron sobre una base **descartable** `iw3_scratch` con la app en el puerto 8089 (`./mvnw spring-boot:run` con `--server.port=8089` y la URL de la base cambiada) y el seed `docs/sql/seed-dev-users.sql`. Al terminar se bajó la app y se borró la base.

7.1 Tests de Maven

`./mvnw -B test` → **40 tests, 0 fallas, 0 errores** (`BUILD SUCCESS`): `ActivationPasswordGeneratorTest` 2, `OrderBusinessActivateTest` 6, `OrderBusinessCloseTest` 7, `OrderBusinessFinalWeightTest` 11, `OrderBusinessTareTest` 4, `OrderDetailValidatorTest` 9, `Iw3ApplicationTests` 1. Son los 19 previos más los 21 de la Parte 1. La Parte 1 comprobó además que, con el código anterior a los arreglos, `OrderBusinessTareTest` falla 1 de 4 y `OrderBusinessActivateTest` 5 de 6 (los tests detectan los defectos).

Test (`OrderBusinessFinalWeightTest`)	Qué comprueba
`registerFinalWeight_orderNotFound_throwsNotFound`	404; no guarda ni escribe historial
`registerFinalWeight_zeroNegativeNaNAndInfinite_throwBadRequest`	0, negativo, `NaN` e infinito → 400; nada se escribe
`registerFinalWeight_statusNot3_throwsConflict` · `registerFinalWeight_noTare_throwsConflict`	Estados 1, 2 y 4 → 409 · tara nula → 409
`registerFinalWeight_weightLessThanTare_throwsBadRequest`	Menor que la tara → 400
`registerFinalWeight_valid_movesTo4SavesWeightAndLogsOneRow`	Ejemplo de la división: estado 4, peso y fecha guardados, búsqueda con bloqueo, **una** fila 3 → 4 y la conciliación 14500 / 16490 / 2000 / 1990 / −10 / 21 / 0.5 / 3300
`registerFinalWeight_nullOrBlankActor_logsSistema`	Actor `null` o en blanco → `sistema`
`registerFinalWeight_weightEqualToTareWithoutReadings_isValid`	Peso igual a la tara y sin lecturas: válido, 0.0 y promedios `null`
`reconciliation_orderNotFound_throwsNotFound` · `reconciliation_statusNot4_throwsConflict` · `reconciliation_status4_returnsTheSameValuesWithoutLockingNorWriting`	404 · 409 en los estados 1, 2 y 3 · en el estado 4 devuelve los valores sin bloquear ni escribir

7.2 Postman con newman

```
npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json --env-var "baseUrl=http://localhost:8089"
```

Resultado: **151 pedidos, 358 aserciones, 0 fallas** (18 s).

Carpeta	Pedidos	Aserciones	Fallas
00 a 04 (línea base)	86	204	0
05 - Fase 5: pesaje final y conciliación (TMS)	44	98	0
06 - Circuito completo (1 a 4)	21	56	0
Total	**151**	**358**	**0**

También se corrieron solas: `--folder "00 - Autenticación" --folder "05 …"` → 46 pedidos y 102 aserciones; `--folder "06 …"` **sin** la carpeta 00 → 21 pedidos y 56 aserciones, 0 fallas (la 06 hace su propio login).

Carpeta 05 (variables `orderNumberFinal`, `activationPasswordFinal`, `reconciliationFinal`, `orderNumberFinalPending`, `orderNumberFinalTared`, `orderNumberFinalBare`, `reconciliationFinalBare`, `orderNumberFinalTare`):

Grupo	Pedidos y resultado
Preparación	Crear (201) · tara 14500 (200) · activar (200, preset 15000) · lecturas 1000/0.5/20/3000 y 2000/0.5/22/3600 (`saved: true`) · cerrar (200, estado 3)
Pesaje final inválido	`{}` , 0, −100 → 400 «mayor que 0» · `1e999` → 400 «finito» con `code` 400 · 14000 → 400 «menor que la tara» y «14500» · la orden sigue en estado 3 con `finalWeight` y fecha `null` · historial con **3** filas
Errores	Inexistente (`1999999999`) → 404 · sin token → 401 · conciliación en estado 3 → 409
Pesaje final	16490.0 → 200: `tare` 14500, `finalWeight` 16490, `loadedProduct` 2000, `scaleNet` 1990, `difference` −10, promedios 21 / 0.5 / 3300 (`closeTo` 1e-9) · orden en estado 4 con `finalWeight` y `finalWeighingDateTime` , sin `activationPassword` · historial con 4 filas, la última 3 → 4, `admin` , «Pesaje final registrado» · GET = POST
Estado 4 es final	Segundo pesaje (409 «no admite pesaje final», «estado 4») · lectura (409 «no acepta lecturas») · activación (409 «no está lista para activar») · cierre (409 «no se puede cerrar») · tara (409 «no está pendiente») · la conciliación no cambió
Fuera de estado	Conciliación en estado 1 y 2 → 409 · inexistente → 404 · sin token → 401 · pesaje final en estado 1 y 2 → 409 · **peso 0 sobre una orden en estado 1 → 400** (D2: el peso se valida antes que el estado; la Parte 1 lo había probado solo con Mockito)
Límite (D2, D6)	Orden sin activar ni lecturas, tara 14500, cierre (estado 3, `lastAccumulatedMass` `null`) · pesaje 14500 → 200 con 0 / 0 / 0 y promedios `null` · GET = POST
Regresión H-26	Tara `1e999` → 400 «finito» con `code` 400 · la orden sigue en estado 1 con `tare` `null`

Carpeta 06 (variables `tokenCircuit`, `orderNumberCircuit`, `presetCircuit`, `tareCircuit`, `activationPasswordCircuit`, `readingsCircuit`, `finalWeightCircuit`, `reconciliationCircuit`; autenticación Bearer propia a nivel de carpeta):

1. Login → `tokenCircuit` .
2. **Punto 1:** crear (201, `location`) · estado 1 con `receptionDateTime` y el preset · reenviar → **302** («Se encontró la Orden»; sin seguir la redirección).
3. **Punto 2:** tara 14500.5 → contraseña de 5 dígitos · estado 2 con la tara y `initialWeighingDateTime` , sin contraseña expuesta.
4. **Punto 3:** activación → preset 15000 · 4 lecturas válidas (1200.5, 2400.25, 3600.75 y 4800.1 kg, con densidades, temperaturas y caudales no enteros) y **una descartada** (3000 kg, menor que la anterior: `saved: false` con motivo) · cabecera = última lectura válida, con activación, inicio y fin de carga.
5. **Punto 4:** cierre → estado 3 · lectura tardía → 409.
6. **Punto 5:** pesaje final 19310.35 → el test **recalcula** la conciliación con la tara, el peso y las lecturas guardadas en `readingsCircuit` y compara los 8 valores con `closeTo` 1e-6 · `GET …/reconciliation` = POST · estado 4 con `finalWeight` y fecha.
7. Historial con **exactamente** `[[null,1],[1,2],[2,3],[3,4]]` y el usuario en cada fila · la orden aparece en `GET /orders` (si no está en la primera página, el test recorre las siguientes con `pm.sendRequest`) · `GET /orders/{n}/details` tiene las 4 lecturas válidas, en orden.

7.3 Concurrencia (criterios 12 y 14 y regresión de la Fase 4)

Script descartable de bash y `curl` (fuera del repo) contra `iw3_scratch` . Cada caso se verificó con `SELECT` sobre la base.

Nota de honestidad: la primera corrida del script tenía dos errores propios (el número de orden no avanzaba porque se calculaba en una *subshell*, y la contraseña no se extraía porque la respuesta JSON viene en varias líneas). Esas corridas se descartaron; los resultados de abajo son de la corrida corregida, completa.

Prueba	Repeticiones	Resultado
(a) Dos pesajes finales simultáneos sobre una orden en estado 3	5	Siempre **un 200 y un 409**; estado 4; **1** fila 3 → 4 en cada orden
(b) Activación y cierre simultáneos sobre una orden en estado 2 sin activar (H-25)	20	18 × (activación 200, cierre 200) y 2 × (activación 409, cierre 200). **Estado final 3 en las 20**; 1 fila 2 → 3; cuando la activación ganó, `activation_date_time` quedó guardada; cuando perdió, `NULL`. Ninguna orden quedó en estado 2
(c) Dos activaciones simultáneas (H-16)	5	Ambas **200** en las 5; con el *general log* de MySQL activado, **un solo** `UPDATE orders` **con** `activation_date_time` por orden
(d) 50 lecturas y un cierre en paralelo (regresión de la Fase 4)	2	Corrida 1 y 2: 25 × 200 y 25 × 409, **0 respuestas 500**, cierre 200, estado 3, 1 fila 2 → 3, cabecera 550 = `MAX(accumulated_mass)` 550, 28 filas de detalle (3 previas + 25), 0 lecturas con estampa posterior al cierre

El log de la app no registró ningún `ERROR`, *deadlock* ni espera de bloqueo vencida. Hubo una pausa de ~1 min 47 s entre dos repeticiones de (b), que el pool de conexiones informó como `Thread starvation or clock leap detected`: fue una pausa del proceso o del equipo, no de la base (las respuestas de esa repetición fueron 200/200 con estado final 3).

7.4 Casos límite (a mano, con `curl`)

Caso	Resultado
`{"weight": "abc"}`	400 con el formato por defecto de Spring (H-21)
`{"weight": "16490"}` (número como texto)	Se acepta (llegó a la búsqueda de la orden: 404 sobre `1999999999`)
Sin cuerpo	400 de Spring («Required request body is missing»)
`…/orders/abc/final-weight`, `/orders/abc/reconciliation`	400 de Spring (H-23)
`Content-Type: text/plain`	415

7.5 Swagger

`/v3/api-docs` publica `POST …/final-weight` (200 con esquema `Reconciliation`, 400, 404, 409) en el grupo de la balanza y `GET …/reconciliation` (200, 404, 409). Los esquemas `Reconciliation` (9 campos), `FinalWeightRequest` y `Order` (con `finalWeight` y `finalWeighingDateTime`) figuran. La interfaz gráfica no se recorrió a mano: se comprobó el documento OpenAPI, que es lo que muestra.

8. Cambios de comportamiento respecto de las fases anteriores

Antes	Ahora	Por qué
`POST …/tare` con `{"weight": 1e999}` → 500 con el formato de Spring y la orden en estado 1	**400** `weight debe ser un número finito` (`StandartResponse`), orden en estado 1 con tara nula	D13 (H-26)
`activate` buscaba la orden sin bloqueo	La busca con `SELECT … FOR UPDATE`: se serializa con el cierre, con las lecturas y con otra activación de la misma orden	D14 (H-25, H-16)
`FINISHED` (4) no lo usaba ninguna operación	Lo produce el pesaje final; desde el 4, las operaciones de los Puntos 2 a 5 dan 409 (sin cambiar su código)	Punto 5
`GET /orders/{n}`	Suma `finalWeight` y `finalWeighingDateTime` (nulos hasta el pesaje final)	D4

El resto de las respuestas de las Fases 1 a 4 no cambió: las 86 peticiones y 204 aserciones de las carpetas 00 a 04 siguen pasando.

9. Auditoría final del segundo parcial

Auditoría hecha por Benjamín Alasino sobre el código de la rama (`0622a20` y anteriores) contra el PDF oficial. Clasificación: **IV** implementado y verificado · **INV** implementado y no verificado · **P** parcial · **D** con defectos · **Pend** pendiente.

9.1 Puntos 1 a 5 contra el PDF

Requisito (PDF)	Archivo y método	Prueba	Resultado	Clase
P1: recibir número, camión, chofer, cliente, producto, fecha prevista y preset; entidades por código externo (string)	`OrderSapRestController.add` → `OrderBusiness.add`, `OrderValidator`, `sync*`	Postman 01 y 06	201; 302 al repetir; 400 con datos inválidos	IV
P1: orden creada en estado 1	`OrderBusiness.add`	Postman 01, 06	Estado 1 y fila `null → 1`	IV
P2: tara (balanza) y contraseña de 5 dígitos asociada; estado 2	`OrderTmsRestController.registerTare` → `OrderBusiness.registerTare`, `ActivationPasswordGenerator`	Postman 02, 06; `ActivationPasswordGeneratorTest`	200 con 5 dígitos; estado 2; fila 1 → 2	IV
P3: contraseña + número de orden; solo si existe, está en el estado esperado y la contraseña es correcta → preset	`OrderLoaderRestController.activate` → `OrderBusiness.activate`	Postman 03, 06; `OrderBusinessActivateTest`	200 con preset; 404, 409, 403, 400	IV
P3: datos de detalle (masa, densidad, temperatura, caudal) y descarte (caudal ≤ 0, masa ≤ 0 o menor que la anterior)	`OrderDetailBusiness.add`, `OrderDetailValidator`	Postman 03, 06; `OrderDetailValidatorTest`	Descartes con `saved: false`	IV
P3: cabecera con el último valor válido y estampa; detalle con el histórico	`OrderDetailBusiness.add`	Postman 03, 04, 06	Cabecera = última lectura válida; `GET …/details`	IV
P3: frecuencia de almacenamiento distinta de la de recepción	—	—	Se guardan todas las lecturas válidas (permitido: "es válido almacenar todos los registros")	Pend (opcional, D12)
P4: endpoint de cierre que bloquea nuevos detalles; estado 3	`OrderLoaderRestController.close` → `OrderBusiness.close`	Postman 04, 06; concurrencia (d)	200; lectura posterior 409; fila 2 → 3	IV
P5: recibir el pesaje final y pasar a estado 4	`OrderTmsRestController.registerFinalWeight` → `OrderBusiness.registerFinalWeight`	Postman 05, 06; concurrencia (a)	200; estado 4; fila 3 → 4	IV
P5: devolver la conciliación con tara, pesaje final, producto cargado (última masa), neto (final − inicial), diferencia (neto − producto) y promedios de temperatura, densidad y caudal según datos almacenados	`OrderBusiness.buildReconciliation`, `OrderDetailRepository.averagesByOrder`, `Reconciliation`	Postman 05 (valores exactos) y 06 (recalculados en el test)	Los 8 valores coinciden; campos `tare`, `finalWeight`, `loadedProduct`, `scaleNet`, `difference`, `averageTemperature`, `averageDensity`, `averageFlowRate`	IV
P5: conciliación en cualquier momento, solo en estado 4, con endpoint específico	`OrderRestController.reconciliation` → `OrderBusiness.reconciliation`	Postman 05, 06	200 en estado 4; 409 en 1, 2 y 3	IV

Requisito (PDF)	Archivo y método	Prueba	Resultado	Clase
Anexo: fechas de recepción, pesaje inicial, inicio y fin de carga y **pesaje final** ("momento de recepción")	`Order` (`receptionDateTime`, `initialWeighingDateTime`, `loadingStartDateTime`, `loadingEndDateTime`, `finalWeighingDateTime`)	Postman 06 (todas presentes)	Hora del servidor en todos los casos	IV
Anexo: entidades con código externo y atributos (patente, cisternado, nombre, apellido, razón social...)	`Truck`, `TruckTank`, `Driver`, `Customer`, `Product`	Postman 01	Sincronizadas por `externalCode`	IV
Notas: estados secuenciales y unidireccionales	Comprobación de estado en `registerTare`, `activate`, `OrderDetailBusiness.add`, `close`, `registerFinalWeight`	Postman 02 a 05 (409 en cada transición inválida); estado 4 final	Ninguna operación retrocede un estado	IV
Notas: cada cambio registrado con fecha y hora, usuario o proceso y observaciones	`OrderBusiness.logStatus` → `order_status_logs`	Postman 06 (4 filas con usuario y fecha)	`null→1`, `1→2`, `2→3`, `3→4`	IV

9.2 Requisitos del segundo parcial (PDF, p. 8)

Requisito	Evidencia	Clase
APIs de los Puntos 1 a 5 **sin interfaz gráfica**	7 endpoints REST de integración y consulta; el proyecto no tiene vistas	IV
Documentación **OpenAPI/Swagger**	`/v3/api-docs` y `/swagger-ui.html`. Los 11 endpoints del TP tienen `@Operation`. Los de los Puntos 1 a 5 declaran sus respuestas con `@ApiResponse`	IV para los Puntos 1 a 5; **P** para la cobertura total (H-27)
Colección Postman que pruebe todas las APIs	Cruce de los 13 endpoints de `/v3/api-docs` contra la colección (tabla 9.3): 12 cubiertos; falta `GET /demo/encodepass` (demo de la cátedra, H-28)	IV para las APIs del TP; **P** si se cuenta el demo
Persistencia	MySQL; las pruebas consultaron las tablas con `SELECT`	IV
Opcional: circuito completo con *Tests* de Postman	Carpeta 06 (el PDF dice "hasta el cierre"; la 06 llega al estado 4)	IV
Opcional: promedios	`AVG` (decisión 3, D5)	IV
Opcional: frecuencia de almacenamiento	No implementada (D12)	Pend (opcional)

9.3 Endpoints de `/v3/api-docs` contra la colección Postman

Método y ruta	`@Operation`	Respuestas declaradas	Pedidos	Carpetas
`POST /api/v1/login`	No (`AuthRestController`, cátedra)	200	3	00, 06
`POST /api/v1/integration/sap/orders`	Sí	201/302/400	19	01 a 06
`POST /api/v1/integration/tms/orders/{n}/tare`	Sí	200/400/404/409	18	02 a 06

Método y ruta	`@Operation`	Respuestas declaradas	Pedidos	Carpetas
`POST /api/v1/integration/loader/activation`	Sí	200/400/403/404/409	14	03 a 06
`POST /api/v1/integration/loader/orders/{n}/details`	Sí	200/400/404/409	31	03 a 06
`POST /api/v1/integration/loader/orders/{n}/close`	Sí	200/401/404/409/500	10	04 a 06
`POST /api/v1/integration/tms/orders/{n}/final-weight`	Sí	200/400/404/409	14	05, 06
`GET /api/v1/orders`	Sí	200	1	06 (antes de esta fase no lo probaba ninguna carpeta)
`GET /api/v1/orders/{n}`	Sí	200 (no declara 404)	17	01 a 06
`GET /api/v1/orders/{n}/status-log`	Sí	200 (no declara 404)	7	01 a 06
`GET /api/v1/orders/{n}/details`	Sí	200/404	8	03, 04, 06
`GET /api/v1/orders/{n}/reconciliation`	Sí	200/404/409	9	05, 06
`GET /demo/encodepass`	No (demo de la cátedra, público)	200	**0**	—

9.4 Decisiones D1 a D14 contra el código

D	Comprobación en el código	Cumple
D1	`registerFinalWeight`: `status != CLOSED_FOR_LOADING` → 409; el segundo pesaje encuentra el estado 4 → 409 (Postman y concurrencia (a))	Sí
D2	Orden en el código: `loadForUpdate` (404) → `requireValidWeight` (400) → estado (409) → tara nula (409) → `weight < tare` (400). Usa `<`: igual es válido (Postman límite: 200). El 400 de la tara va **después** del 409; peso 0 en estado 1 → 400 (Postman)	Sí
D3	`@Transactional(rollbackFor = Exception.class)`; `loadForUpdate`; `saveAndFlush` dentro del `try`; `reconciliation` usa `load` y no es transaccional	Sí
D4	Solo dos columnas nuevas en `Order`; `Reconciliation` no es entidad; `buildReconciliation` es la única que calcula y la llaman el POST y el GET	Sí
D5	`scaleNet = finalWeight - tare`, `difference = scaleNet - loadedProduct`, `AVG` en JPQL; ningún `Math.round` ni `BigDecimal`	Sí
D6	`lastAccumulatedMass != null ? … : 0.0`; promedios `null` si `avg == null` o sus valores son nulos; no hay chequeo que rechace órdenes sin lecturas	Sí
D7	POST devuelve `Reconciliation` con 200; GET 200/404/409	Sí
D8	`logStatus(order, from, FINISHED, actor, "Pesaje final registrado")`; ninguna operación sale del estado 4	Sí
D9	`@Autowired OrderDetailRepository orderDetailDAO` en `OrderBusiness` (no `IOrderDetailBusiness`); `Constants` sin cambios	Sí
D10	Los 7 textos de la sección 3.6 coinciden con el código (verificados por `include` en Postman)	Sí
D11	Variables propias en 05 y 06; diff del JSON: 0 líneas borradas	Sí
D12	No hay frecuencia de almacenamiento, roles, alarmas ni frontend	Sí
D13	`registerTare` llama a `requireValidWeight`; el mensaje «mayor que 0 (kg)» se mantiene (carpeta 02 pasa)	Sí

D	Comprobación en el código	Cumple
D14	`activate` : el `if (orderNumber == null	

9.5 Otros controles

Control	Resultado
Comprobaciones fuera del `try`	En `registerFinalWeight` , `reconciliation` , `registerTare` , `activate` y `close` ningún `throw` de 400/404/409/403 está dentro de un `try` con `catch (Exception)` . El 404 de `loadForUpdate` también va fuera de su `try`
Lógica duplicada	Una sola `buildReconciliation` ; `requireValidWeight` compartida; `loadForUpdate` reutilizado (sigue el duplicado ya registrado con `OrderDetailBusiness` , H-24, sin un tercero)
Dependencias nuevas	`pom.xml` sin cambios
Lista de detalles en `Order`	No hay `@OneToMany` ni `List` en `Order`
Cambios fuera de lo permitido en las Fases 1 a 4	El diff de `OrderBusiness` frente a `main` solo cambia la validación del peso en `registerTare` (D13) y `load` → `loadForUpdate` en `activate` (D14)
Contraseña en mensajes, logs y consultas	El 403 dice «La contraseña de activación es incorrecta» (sin números). `GET /orders/{n}` no la expone (`@JsonIgnore` , Postman 05 y 06). Se buscaron en el log de la app **todas** las contraseñas generadas en `iw3_scratch` : **0 apariciones** (`show-sql` imprime `?` , no los valores)
Regresiones de las Fases 1 a 4	Carpetas 00 a 04: 86 pedidos y 204 aserciones, 0 fallas; 19 tests previos en verde; concurrencia (d) igual a la de la Fase 4
Estado 4 final	Lectura, activación, cierre, tara y pesaje final → 409 (Postman 05)
Columnas sobre una base con datos	Parte 1: clon de `iw3_db` con una orden, columnas agregadas sin error. Hoy en `iw3_db` : ambas nulables y la orden previa intacta

9.6 Conclusión de la auditoría

- **Fase 5: completa.** Los 21 criterios están cumplidos y verificados.
- **Segundo parcial, requisitos obligatorios:** los Puntos 1 a 5 están implementados y verificados como API sin interfaz gráfica, con Swagger y con una colección Postman que prueba todas las APIs del TP. Las dos observaciones parciales (H-27: anotaciones de Swagger fuera de los Puntos 1 a 5 y respuestas 401/500 no declaradas; H-28: el demo `/demo/encodepass` sin pedido en Postman) **no** afectan a los endpoints de los Puntos 1 a 5 y se dejan para que el equipo decida.
- **Opcionales:** circuito con *Tests* y promedios hechos; frecuencia de almacenamiento no hecha (D12).
- **Defectos que bloqueen:** ninguno. No se hizo ningún cambio de código en la auditoría.

10. Hallazgos y limitaciones

ID	Tipo	Descripción	Evidencia	Estado / a quién corresponde
H-25	Limitación de la Fase 4	`activate` sin bloqueo podía deshacer un cierre	Concurrencia (b): 20 de 20 con estado final 3	**Resuelto** por D14 (Calvo)
H-16	Limitación de la Fase 3	Activaciones simultáneas de la misma orden	Concurrencia (c): 1 `UPDATE` por orden	**Resuelto** como efecto colateral de D14
H-26	Defecto de la Fase 2	Tara infinita → 500 con el formato de Spring	Postman 05 «Regresión H-26»: 400	**Resuelto** por D13 (Calvo)

ID	Tipo	Descripción	Evidencia	Estado / a quién corresponde
H-12	Limitación de la Fase 2	Taras simultáneas sobre la misma orden (`registerTare` busca sin bloqueo)	Código	**Sigue aceptado** para el parcial (fuera de alcance, D13 solo cambia el peso)
H-27	Observación (documentación)	Swagger incompleto **fuera** de lo nuevo: `POST /login` y `GET /demo/encodepass` (cátedra) sin `@Operation`; `GET /orders/{n}` y `/status-log` no declaran el 404 que devuelven; ningún endpoint salvo el cierre declara 401 y 500 (tampoco `final-weight` ni `reconciliation`)	`/v3/api-docs` (tabla 9.3)	Abierto. Arreglo propuesto: solo anotaciones `@ApiResponse` (y `@Operation` en el login). Corresponde a los autores de cada endpoint o a quien el equipo designe; no se toca sin autorización
H-28	Observación (Postman)	`GET /demo/encodepass` (demo público de la cátedra) no tiene pedido en la colección	Tabla 9.3	Abierto. Propuesta: retirarlo o cerrarlo junto con H-5 (no es una API del TP); si el equipo prefiere conservarlo, agregar un pedido (Postman es de la Parte 2)
H-29	Observación (Fase 2)	`registerTare` sigue guardando con `save` y no con `saveAndFlush` : una falla de base al confirmar saldría como 500 de Spring (la lección de H-14). Con D13 el caso del peso infinito ya no llega a la base	Código	Abierto, deuda menor. Fuera de alcance (D13: "el resto de `registerTare` no cambia"); cambio de una línea si el equipo lo autoriza
H-21, H-23	Observaciones previas	`weight` como texto no numérico, cuerpo ausente u `orderNumber` no numérico → 400 con el formato de Spring; un número como texto se acepta	Sección 7.4	Sin cambios
—	Limitación	El token dura 1 h (H-18) y no hay roles: cualquier usuario autenticado puede registrar un pesaje final	—	Examen final

11. Pendientes

- **Pull Request** de la rama a `main` como borrador, cuando lo pida Benjamín; aprobaciones de Martín, Contreras y Calvo (decisión 14). El *ruleset* de `main` existe pero está deshabilitado: el acuerdo se cumple a mano.
- **H-27, H-28 y H-29:** decidir en equipo (ninguno bloquea el parcial).
- **Decisión 30** (cierre sin activación) sigue figurando como "a confirmar por el equipo"; la Fase 5 la acompaña con D6.
- **Examen final:** roles, alarma de temperatura, frontend, seguridad (H-5) y, como opcional, la frecuencia de almacenamiento.

Parte 2 — Guía de estudio de la Fase 5

A. Objetivo de la fase

Cerrar el ciclo de la orden. Cuando el camión cargado vuelve a la balanza, el TMS envía el **pesaje final**: la orden pasa del estado 3 (*cerrada para carga*) al 4 (*finalizada*) y el backend responde la **conciliación**, que compara lo que dice la balanza (final − tara) con lo que midió el caudalímetro (última masa acumulada). La conciliación se puede volver a pedir en cualquier momento, solo para órdenes finalizadas. Además, la fase prueba el **circuito completo 1 → 4** y audita que el segundo parcial esté completo.

B. Conceptos teóricos

1. Conciliación (dominio de la consigna)

Dos instrumentos miden lo mismo de formas distintas: la **balanza** pesa el camión vacío y lleno (neto por balanza = final − tara) y el **caudalímetro másico** acumula los kilos que pasaron por la bomba (producto cargado = última masa acumulada). La **diferencia** (neto − producto) muestra si alguno de los dos está descalibrado o si hubo una pérdida. Los promedios de temperatura, densidad y caudal describen cómo fue la carga.

2. Datos derivados: calcular frente a guardar (complementario)

Un dato que se obtiene de otros ya guardados conviene **calcularlo** al consultarlo: si se guardara, habría dos versiones de la verdad y podrían divergir. Por eso la conciliación no tiene tabla (D4). Se guarda solo lo que llega de afuera y no se puede reconstruir: el peso final y su hora.

3. Funciones de agregación y proyecciones (clase 7 de la cátedra: consultas; la proyección por interfaz es complementaria)

`AVG` es una función de agregación de SQL: devuelve **una** fila con el promedio de una columna, y `NULL` si no hay filas. Spring Data permite declarar la consulta con `@Query` y recibir el resultado en una **interfaz de proyección** (`ReadingAverages`): Spring crea un objeto que implementa los getters con los alias de la consulta (`as temperature` → `getTemperature()`). El promedio es **simple**: cada lectura guardada pesa igual, aunque haya llegado con más o menos tiempo de diferencia.

4. Bloqueo pesimista y transacciones (clases 3 y 7; el bloqueo es complementario)

`findByOrderNumberForUpdate` ejecuta `SELECT … FOR UPDATE` y bloquea la fila hasta el `COMMIT`. Dos pesajes finales de la misma orden se procesan de a uno: el segundo ve el estado 4 y recibe 409. `@Transactional(rollbackFor = Exception.class)` hace que el `UPDATE` de la orden y el `INSERT` del historial se confirmen juntos (las excepciones de negocio son *checked* y sin `rollbackFor` Spring no revertiría). D14 aplicó el mismo bloqueo a la activación.

5. Valores de punto flotante (complementario)

`double` no representa exactamente todos los decimales: `18400.2 - 14500.1` da `3900.1000000000004`. Por eso el backend **no redondea** (redondear es tarea de quien muestra el dato) y Postman compara con `closeTo` (una tolerancia), no con igualdad. El ejemplo de la división usa valores exactos (múltiplos de potencias de 2) para que la comparación sea exacta. `1e999` es un número válido en JSON, pero se convierte en `Infinity`: por eso hace falta `Double.isInfinite`.

6. Tests de Postman encadenados (complementario: la cátedra no usa Postman)

Cada pedido tiene un *pre-request script* (antes de enviar) y un script de *tests* (después). Con `pm.collectionVariables.set` un pedido guarda lo que necesita el siguiente (número de orden, contraseña, lecturas enviadas), y `pm.test` + `pm.expect` hacen las aserciones. newman ejecuta la colección por consola. La carpeta 06 encadena así los cinco puntos y **recalcula** la conciliación en el test.

C. Arquitectura

```
Balanza (TMS) ── POST /api/v1/integration/tms/orders/{n}/final-weight { weight } + JWT
        │
        ▼
JWTAuthorizationFilter ── sin token → 401
        │
        ▼
OrderTmsRestController.registerFinalWeight (getUserLogged → actor; try/catch → 400/404/409/500)
        │
        ▼
OrderBusiness.registerFinalWeight   @Transactional(rollbackFor = Exception.class)
        │   loadForUpdate ─────► OrderRepository.findByOrderNumberForUpdate   (SELECT … FOR UPDATE)
        │   requireValidWeight · estado 3 · tara · weight ≥ tare
        │   saveAndFlush ──────► UPDATE orders (final_weight, final_weighing_date_time, status = 4)
        │   logStatus ─────────► INSERT order_status_logs (3 → 4)
        │   buildReconciliation ► OrderDetailRepository.averagesByOrder (SELECT avg(…))
        ▼
200 { orderNumber, tare, finalWeight, loadedProduct, scaleNet, difference, averageTemperature, averageDensity,
averageFlowRate }

Cualquier cliente ── GET /api/v1/orders/{n}/reconciliation ──► OrderRestController.reconciliation
        ──► OrderBusiness.reconciliation (load sin bloqueo · estado 4 o 409) ──► buildReconciliation (la misma)
```

D. Código paso a paso

1. El pesaje final

```
@Override
@Transactional(rollbackFor = Exception.class)
public Reconciliation registerFinalWeight(Integer orderNumber, double weight, String actor)
        throws NotFoundException, BadRequestException, ConflictException, BusinessException {
    Order order = loadForUpdate(orderNumber);                       // (1) 404, con bloqueo
    requireValidWeight(weight);                                     // (2) 400: ≤ 0, NaN o infinito
    if (order.getStatus() != OrderStatus.CLOSED_FOR_LOADING) {      // (3) 409
        throw ConflictException.builder().message("La orden " + orderNumber
                + " no admite pesaje final: debe estar en estado 3 (estado " + order.getStatus().getCode() +
")").build();
    }
    if (order.getTare() == null) { … 409 … }                        // (4) defensa
    if (weight < order.getTare()) { … 400 … }                       // (5) después del estado: la tara existe
    try {
        OrderStatus from = order.getStatus();                       // (6) 3, para el historial
        order.setFinalWeight(weight);
        order.setFinalWeighingDateTime(LocalDateTime.now());        // (7) hora del servidor
        order.setStatus(OrderStatus.FINISHED);
        order = orderDAO.saveAndFlush(order);                       // (8) UPDATE dentro del try
        logStatus(order, from, order.getStatus(), actor, "Pesaje final registrado");
        return buildReconciliation(order);                          // (9) la única función de cálculo
    } catch (Exception e) {
        log.error(e.getMessage(), e);
        throw BusinessException.builder().ex(e).build();            // (10) 500 y rollback
    }
}
```

- **(1) a (5) fuera del `try`:** si no, el `catch (Exception)` los convertiría en 500.
- **(5) después de (3):** en el estado 1 la tara es `null` y `weight < null` lanzaría `NullPointerException`.
- **`<` y no `<=`:** peso igual a la tara es válido (orden sin carga).

2. La función compartida

```
private Reconciliation buildReconciliation(Order order) {
    Double tare = order.getTare();
    Double finalWeight = order.getFinalWeight();
    Double loadedProduct = order.getLastAccumulatedMass() != null ? order.getLastAccumulatedMass() : 0.0;
    Double scaleNet = finalWeight - tare;
    Double difference = scaleNet - loadedProduct;
    ReadingAverages avg = orderDetailDAO.averagesByOrder(order);
    return new Reconciliation(order.getOrderNumber(), tare, finalWeight, loadedProduct, scaleNet, difference,
            avg == null ? null : avg.getTemperature(), avg == null ? null : avg.getDensity(),
            avg == null ? null : avg.getFlowRate());
}
```

3. La consulta

`reconciliation(orderNumber)`: `load` (404, sin bloqueo) → si el estado no es `FINISHED`, 409 → `buildReconciliation` dentro de un `try`.

4. Los dos arreglos

```
private void requireValidWeight(double weight) throws BadRequestException {
    if (!(weight > 0)) { throw … "weight debe ser mayor que 0 (kg)" … }   // también NaN
    if (Double.isInfinite(weight)) { throw … "weight debe ser un número finito" … }
}
// activate: después de comprobar los nulos
Order order = loadForUpdate(orderNumber);   // antes: load(orderNumber)
```

E. Trazabilidad de los datos

Dato de la conciliación	Origen	Dónde se guarda	Quién lo escribe
`tare`	Balanza, Punto 2	`orders.tare`	`registerTare`
`finalWeight`	Balanza, Punto 5	`orders.final_weight`	`registerFinalWeight`
`loadedProduct`	Caudalímetro, Punto 3	`orders.last_accumulated_mass`	`OrderDetailBusiness.add`
`scaleNet`, `difference`	Calculados	**No se guardan**	`buildReconciliation`
Promedios	Lecturas válidas	`order_details` (se promedian al consultar)	`OrderDetailBusiness.add` / `averagesByOrder`
Fecha del pesaje final	Reloj del servidor	`orders.final_weighing_date_time` y `order_status_logs.date_time`	`registerFinalWeight` / `logStatus`
Usuario	JWT	`order_status_logs.actor`	`logStatus`

F. Ejemplo práctico de punta a punta

1. SAP crea la orden 90001 → estado 1 (`null → 1`).
2. La balanza registra la tara 14500 → estado 2, contraseña 48213 (`1 → 2`).
3. El sistema de carga se activa → preset 15000. Lecturas: 1000 kg (0.5, 20 °C, 3000 kg/h) y 2000 kg (0.5, 22 °C, 3600 kg/h).
4. Cierre → estado 3 (`2 → 3`).
5. La balanza envía `{ "weight": 16490.0 }` → 200:

```
{ "orderNumber": 90001, "tare": 14500.0, "finalWeight": 16490.0, "loadedProduct": 2000.0,
  "scaleNet": 1990.0, "difference": -10.0, "averageTemperature": 21.0,
  "averageDensity": 0.5, "averageFlowRate": 3300.0 }
```

La balanza dice que entraron 1990 kg y el caudalímetro 2000: diferencia −10 kg. La orden queda en estado 4 (`3 → 4`, «Pesaje final registrado»). 6. `GET /api/v1/orders/90001/reconciliation` → la misma respuesta. Un segundo pesaje → 409. Una lectura, activación, cierre o tara → 409.

G. Preguntas del profesor

1. ¿Qué pide el Punto 5 y con qué endpoints lo resolvieron?
2. ¿Cómo se calcula cada valor de la conciliación?
3. ¿Por qué no guardan la conciliación en la base?
4. ¿Por qué el POST y el GET usan la misma función?
5. ¿En qué orden validan el pesaje final y por qué hay un 400 después de un 409?
6. ¿Por qué un peso igual a la tara es válido y uno menor no?
7. ¿Qué pasa con una orden que se cerró sin lecturas?
8. ¿Cómo calculan los promedios? ¿Son ponderados?
9. ¿Por qué no redondean y cómo compara Postman los decimales?
10. ¿Qué pasa si la balanza envía dos pesajes finales al mismo tiempo? ¿Cómo lo probaron?
11. ¿Qué era H-26 y cómo se arregló?
12. ¿Qué era H-25 y por qué el arreglo también resolvió H-16?
13. ¿Por qué `OrderBusiness` inyecta `OrderDetailRepository` y no `IOrderDetailBusiness`?
14. ¿Qué hace la carpeta 06 de Postman y cómo encadena los pedidos?
15. ¿Cómo verificaron que el segundo parcial está completo? ¿Qué quedó observado?
16. ¿Qué pasa con una orden en estado 4 si alguien intenta operar sobre ella?

H. Respuestas modelo

1. **Punto 5.** Recibir el pesaje final, pasar a estado 4 y devolver la conciliación; consultarla solo en estado 4. `POST /api/v1/integration/tms/orders/{n}/final-weight` (balanza) y `GET /api/v1/orders/{n}/reconciliation` (consulta).
2. **Cálculos.** Producto cargado = última masa acumulada válida (`lastAccumulatedMass`); neto por balanza = final − tara; diferencia = neto − producto cargado; promedios = `AVG` de temperatura, densidad y caudal de `order_details`. Son las fórmulas literales del PDF.
3. **No se guarda** porque se deriva de datos guardados (tara, peso final, última masa y lecturas). Guardarla duplicaría información; se calcula en cada consulta (D4).
4. **Una sola función** (`buildReconciliation`) garantiza que la respuesta del pesaje y la consulta posterior no puedan diferir. Lo comprobamos en Postman comparando campo a campo.
5. **Orden:** 404 (orden) → 400 (peso ≤ 0, `NaN` o infinito) → 409 (estado 3) → 409 (sin tara) → 400 (menor que la tara). La comparación con la tara necesita la tara, que en el estado 1 es nula: si fuera antes del estado, saldría un `NullPointerException` (500). Todas van fuera del `try`.
6. **Igual a la tara:** el camión no cargó nada (por ejemplo, una orden cerrada sin lecturas, decisión 30); el neto es 0 y la orden tiene que poder finalizar. **Menor:** el camión pesaría menos vacío que lleno, es un error de la balanza (400).
7. **Sin lecturas** (D6): producto cargado 0.0 y promedios `null`; el pesaje final se acepta. Si se rechazara, la orden quedaría para siempre en el estado 3.
8. **Promedios:** `AVG` simple en una consulta JPQL (`averagesByOrder`) con una proyección `ReadingAverages`. No son ponderados por tiempo: cada lectura guardada cuenta igual, como pide "según datos almacenados".

9. **Sin redondeo:** son `double` y pueden traer residuos (`3900.1000000000004`); redondear es del cliente. Postman usa `closeTo` con una tolerancia; el ejemplo usa valores exactos.
10. **Dos pesajes simultáneos:** la orden se busca con `SELECT … FOR UPDATE`; el segundo espera, ve el estado 4 y recibe 409. Lo probamos con un script que dispara dos pesajes en paralelo, 5 veces: siempre un 200 y un 409, y una sola fila 3 → 4.
11. **H-26:** `{"weight": 1e999}` en la tara pasaba `!(w > 0)` (infinito es mayor que 0) y MySQL lo rechazaba al guardar: 500 con el formato de Spring. Se agregó `requireValidWeight`, que también rechaza el infinito con 400, y se usa en la tara y en el pesaje final.
12. **H-25:** `activate` leía la orden sin bloqueo y `saveAndFlush` escribe todas las columnas: si se cruzaba con un cierre, podía volver a escribir el estado 2. Con `loadForUpdate`, activación y cierre se serializan (20 de 20 repeticiones terminaron en estado 3). Dos activaciones simultáneas también se serializan: la segunda ve la fecha ya puesta y no escribe (H-16; un solo `UPDATE` en el *general log*).
13. **Dependencia circular:** `OrderDetailBusiness` ya depende de `IOrderBusiness`; si `OrderBusiness` dependiera de `IOrderDetailBusiness`, Spring no podría crear ninguno de los dos. Se inyecta el repositorio, que no depende de nadie.
14. **Carpeta 06:** hace su propio login y recorre una orden: crear (201) y reenviar (302), tara, activación, 4 lecturas válidas y una descartada, cierre y lectura tardía (409), pesaje final y consulta, historial de 4 filas, listado y lecturas guardadas. Cada pedido guarda en variables lo que usa el siguiente, y el pesaje final recalcula la conciliación con los datos enviados.
15. **Auditoría:** una matriz que cruza cada requisito del PDF con su código, su prueba y su resultado, más el cruce de `/v3/api-docs` con la colección. Todo lo obligatorio de los Puntos 1 a 5 está verificado. Quedaron observaciones fuera del Punto 5: anotaciones de Swagger incompletas (H-27), el demo `/demo/encodepass` sin pedido en Postman (H-28) y la tara con `save` (H-29).
16. **Estado 4 final:** todas las operaciones verifican el estado esperado antes de hacer nada; ninguna acepta el 4. Lecturas, activación, cierre, tara y un segundo pesaje dan 409, sin código nuevo.

I. Errores frecuentes

1. Comparar con la tara antes de comprobar el estado: 500 en el estado 1.
2. Rechazar `weight <= tare`: la orden sin carga queda trabada en el estado 3.
3. Calcular la conciliación en el POST y otra vez, con otro código, en el GET: los resultados pueden divergir.
4. Agregar columnas para los valores calculados.
5. Inyectar `IOrderDetailBusiness` en `OrderBusiness`: el contexto no arranca.
6. Alias de la consulta distintos de los getters de la proyección: los promedios salen `null` siempre.
7. Comparar decimales con igualdad exacta en Postman.
8. Validar solo `weight > 0`: el infinito pasa y explota en la base (H-26).
9. Poner las comprobaciones dentro del `try`.
10. Reutilizar variables de otras carpetas de Postman, o depender del token de la carpeta 00 en la 06.
11. Seguir la redirección del 302 en Postman: hay que desactivar `followRedirects` para ver el 302.

J. Lista de comprobación para rendir

- [ ] Sé citar el Punto 5 y los dos endpoints, con método, ruta y respuestas.
- [ ] Sé calcular a mano la conciliación del ejemplo (14500, 16490, 2000 → 1990 y −10).
- [ ] Explico el orden 404 → 400 → 409 → 409 → 400 y por qué el último 400 va después del 409.
- [ ] Explico por qué la conciliación no se guarda y por qué hay una sola función.
- [ ] Sé qué pasa con una orden sin lecturas (D6) y con peso igual a la tara (D2).

- [ ] Explico `AVG`, la proyección `ReadingAverages` y que el promedio es simple.
- [ ] Explico el bloqueo pesimista en el pesaje final y en la activación (H-25, H-16).
- [ ] Explico H-26 y `requireValidWeight`.
- [ ] Defiendo D1 a D14 (decisiones 33 a 46).
- [ ] Puedo correr `./mvnw -B test` (40 tests) y la colección con newman (151 pedidos, 358 aserciones) y explicar las carpetas 05 y 06.
- [ ] Puedo contar las pruebas de concurrencia y sus resultados.
- [ ] Conozco el resultado de la auditoría y los hallazgos H-27 a H-29.

K. Qué debe poder explicar cada integrante

Integrante	Rol en la fase	Debe poder explicar
Tomás Calvo	Parte 1 (negocio y endpoints)	`registerFinalWeight` línea por línea y el orden de comprobaciones; `buildReconciliation` y `averagesByOrder`; por qué `OrderDetailRepository` directo; los arreglos D13 y D14 y sus tests con Mockito (y que fallaban con el código anterior)
Benjamín Alasino	Parte 2 (pruebas, auditoría y documentación)	Las carpetas 05 y 06 (variables, encadenamiento, `closeTo`, recálculo en el test, `followRedirects`); las cuatro pruebas de concurrencia y qué demuestran; la matriz de auditoría y H-27 a H-29
Martín González	Planeamiento (D1 a D14) y revisión del PR	Por qué cada decisión (sección 2), en especial D2, D4, D6 y D14; el recorrido del POST y del GET; el resultado de la auditoría
Joaquín Contreras	Revisión del PR	El recorrido del POST y del GET, el ejemplo numérico, cómo se prueba con Postman y qué quedó observado para el final

Como la evaluación es individual, los cuatro deben poder responder las preguntas de la sección G, no solo las de su parte.